University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Lab Work 2 - Security – Solutions

Infection Techniques and Polymorphism

Ethical and Legal Disclaimer

The techniques presented in this lab work are studied within a **strictly academic context** in order to understand the mechanisms used by malicious software and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and punishable by criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against Automated Data Processing Systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstructing the operation of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by the LOPMI law no. 2023-22 of January 24, 2023.*

Exercise 1 (Logic Bomb with Temporal Trigger)**.** A *logic bomb* is a malicious program that installs itself in a system and waits for a specific event called a **trigger** to execute its offensive function (the **payload**).

Consider the following C program:

```c
#include <stdio.h>
#include <time.h>

int check_trigger() {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);

    // Trigger condition
    if (t->tm_mon == 3 && t->tm_mday == 1) {
        return 1;
    }
    return 0;
}

void payload() {
```

```c
    printf("=== PAYLOAD ACTIVATED ===\n");
    printf("All your files have been encrypted!\n");
}

void normal_operation() {
    printf("System running normally...\n");
}

int main() {
    if (check_trigger()) {
        payload();
    } else {
        normal_operation();
    }
    return 0;
}
```

1. Identify the *trigger* used in this program and explain precisely how it works. On which date will the payload be activated?
2. Modify the `check_trigger()` function so that the trigger only activates if the program has been executed more than 100 times.
3. Propose two techniques to make the *trigger* harder to detect during static code analysis.
4. From an antivirus perspective, what techniques could be used to detect this type of malicious program?

Course Reminder

Logic bomb: a malicious program that remains inactive (dormant) until a specific condition is met. Unlike a virus, a logic bomb does not replicate: it waits for its *trigger* then executes its *payload*.

Common trigger types:

- **Temporal:** specific date or time (e.g., April 1st, Friday the 13th)
- **Counter:** after N executions or N days of use
- **Event-based:** deletion of a user account, no login for N days
- **Network:** receipt of a specific packet, contact with a C&C server
- **Environmental:** presence/absence of a file, process, or registry key

Historical examples:

- **Omega Engineering (1996)** – Timothy Lloyd, a fired system administrator, had planted a logic bomb that deleted the company's design and manufacturing programs 20 days after his departure. Estimated damages: $10 million.
- **Dark Seoul (2013)** – Attack against South Korean banks and media. The malware waited for a specific date (March 20, 2013 at 2:00 PM) to wipe the hard drives of thousands of machines simultaneously.

Answer 1. *1.* ***Trigger identification:***

The trigger is ***temporal****. It uses the* `time()` *and* `localtime()` *functions to get the current date, then checks two conditions:*

- `t->tm_mon == 3`*: the month equals 3*
- `t->tm_mday == 1`*: the day of month equals 1*

> ***Warning***
>
> *In the C language* `struct tm` *structure, the* `tm_mon` *field is indexed* ***starting from 0*** *(January = 0, February = 1, . . . , December = 11). This is a classic pitfall! Therefore* `tm_mon == 3` *corresponds to* ***April*** *(not March). The payload will be activated on* ***April 1st*** *(April Fools' Day!).*

Complete table of `struct tm` ***fields:***

Field	***Meaning***	***Range***	***Note***
`tm_sec`	*Seconds*	*0–60*	*60 for leap second*
`tm_min`	*Minutes*	*0–59*	
`tm_hour`	*Hours*	*0–23*	
`tm_mday`	*Day of month*	*1–31*	*Starts at 1*
`tm_mon`	*Month*	*0–11*	***0 = January***
`tm_year`	*Year*	*since 1900*	***Add 1900***
`tm_wday`	*Day of week*	*0–6*	*0 = Sunday*
`tm_yday`	*Day of year*	*0–365*	*0 = January 1st*
`tm_isdst`	*Daylight saving*	*>0, 0, <0*	*Positive = DST active*

2. ***Modification for execution counter:***

```c
int check_trigger() {
    int count = 0;
    FILE *f = fopen("/tmp/.counter", "r");
    if (f) {
        fscanf(f, "%d", &count);
        fclose(f);
    }

    count++;

    f = fopen("/tmp/.counter", "w");
    if (f) {
        fprintf(f, "%d", count);
        fclose(f);
    }

    return (count > 100) ? 1 : 0;
}
```

Notes:

- *The file* `/tmp/.counter` *stores the counter persistently*

- *The "." prefix makes the file hidden on Linux*
- *The path `/tmp/` is writable by all users*

3. ***Trigger obfuscation techniques:***

   Technique 1: Constant encryption

   Instead of writing values 3 and 1 directly, store them encrypted and decrypt at runtime:

```
int month_trigger = 0x41 ^ 0x42;  // 3, but not directly visible
int day_trigger = 0x43 ^ 0x42;    // 1, but not directly visible
```

   Technique 2: Indirect calculations

   Use arithmetic expressions to mask values:

```
if (t->tm_mon == (15 / 5) && t->tm_mday == (10 - 9))
```

   Other possible techniques:

   - *Store the target date in an external configuration file*
   - *Use environment variables*
   - *Retrieve the date from a remote server*

4. ***Antivirus detection techniques:***

   - ***Behavioral analysis:*** *Monitor calls to `time()`, `localtime()` followed by conditional branching*
   - ***Heuristic detection:*** *Identify "dormant code" patterns with temporal conditions*
   - ***Sandboxing with time manipulation:*** *Execute the program in a virtual environment while accelerating system time to trigger the payload*
   - ***Static analysis:*** *Search for comparisons on `struct tm` fields*
   - ***Emulation:*** *Simulate program execution with different dates*

Key Takeaway

- *A **logic bomb** is a dormant program that waits for an event (trigger) to execute its payload.*
- *The `tm_mon` field of the `struct tm` structure is indexed starting from **0** (classic C pitfall).*
- *Constant obfuscation (XOR, indirect calculations) complicates static analysis but does not prevent dynamic analysis.*
- *Modern antivirus software combines multiple techniques (static, dynamic, heuristic) to detect logic bombs.*

Exercise 2 (XOR Encryption and Polymorphism)**. Polymorphism** is a technique used by viruses to evade signature-based detection. By encrypting the viral code with different keys for each copy, the virus produces different binary signatures while maintaining the same behavior.

XOR (exclusive or) encryption is often used because it has an interesting property: applying the same operation twice with the same key returns the original value.

Consider the following C program:

```c
#include <stdio.h>
#include <string.h>

void xor_crypt(char *data, int len, char key) {
    for (int i = 0; i < len; i++) {
        data[i] = data[i] ^ key;
    }
}

int main() {
    char payload[] = "FORMAT C: /Y";
    char key = 0x42;

    printf("Original : %s\n", payload);

    // Encryption
    xor_crypt(payload, strlen(payload), key);
    printf("Encrypted: ");
    for (int i = 0; i < strlen(payload); i++) {
        printf("%02X ", (unsigned char)payload[i]);
    }
    printf("\n");

    // Decryption
    xor_crypt(payload, strlen(payload), key);
    printf("Decrypted: %s\n", payload);

    return 0;
}
```

1. Explain mathematically why the XOR operation allows both encryption and decryption using the same function.
2. Mentally execute the program with key `0x42`. What is the first byte of the encrypted payload? (The ASCII code for 'F' is 0x46)
3. Modify the `xor_crypt()` function to use a multi-byte key (e.g., `"SECRET"`) instead of a single byte.
4. From an antivirus perspective:
   (a) Why does this technique make signature-based detection ineffective?
   (b) What alternative technique could the antivirus use to detect this type of polymorphic code?

Course Reminder

The XOR (exclusive or) operation is a bitwise logical operation denoted $\oplus$. It returns 1 if and only if the two input bits are different.

Truth table:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Viral polymorphism: a technique consisting of encrypting the virus body with a different key for each copy. Only the decryption routine remains in plaintext. Thus, each copy of the virus has a different binary signature, making signature-based detection impossible.

Connection to the Vernam cipher (One-Time Pad): XOR is at the heart of the Vernam cipher, the only cryptographic system proven to be **theoretically unbreakable** (Shannon, 1949). Required conditions: the key must be (1) as long as the message, (2) perfectly random, and (3) used only once. In practice, viruses reuse short keys, making the encryption vulnerable to frequency analysis.

Answer 2. 1. ***Mathematical explanation of XOR:***

The XOR operation ($\oplus$) has the following properties:

- ***Associativity:*** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
- ***Commutativity:*** $A \oplus B = B \oplus A$
- ***Identity element:*** $A \oplus 0 = A$
- ***Self-inverse:*** $A \oplus A = 0$ *(every element is its own inverse)*

Proof: $A \oplus K \oplus K = A \oplus (K \oplus K) = A \oplus 0 = A$

Thus, if $C = A \oplus K$ (encryption), then $C \oplus K = A \oplus K \oplus K = A$ (decryption).

The same operation with the same key allows both encryption and decryption.

2. ***Calculation of encrypted bytes:***

First byte:

- *Character 'F' in hexadecimal:* `0x46` = `01000110` *in binary*
- *Key* `0x42` = `01000010` *in binary*

Bitwise XOR calculation:

```
  01000110   (0x46 = 'F')
^ 01000010   (0x42 = key)
----------
  00000100   (0x04)
```

The first encrypted byte is ***0x04*** *(EOT control character – End Of Transmission).*

Complete XOR encryption table for `"FORMAT C: /Y"` ***with key*** `0x42`***:***

Character	*ASCII (hex)*	*Key (hex)*	*XOR (hex)*	*XOR (decimal)*
F	0x46	0x42	0x04	4
O	0x4F	0x42	0x0D	13
R	0x52	0x42	0x10	16
M	0x4D	0x42	0x0F	15
A	0x41	0x42	0x03	3
T	0x54	0x42	0x16	22
(space)	0x20	0x42	0x62	98
C	0x43	0x42	0x01	1
:	0x3A	0x42	0x78	120
(space)	0x20	0x42	0x62	98
/	0x2F	0x42	0x6D	109
Y	0x59	0x42	0x1B	27

Complete encrypted result: `04 0D 10 0F 03 16 62 01 78 62 6D 1B`

Warning

Beware of common XOR calculation errors: you must operate bit by bit, not byte by byte. For example, `0x20` $\oplus$ `0x42` $=$ `0x62` *(and not* `0x22` *which would be a simple addition).*

3. **Function with multi-byte key:**

```c
void xor_crypt(char *data, int len, const char *key) {
    int key_len = strlen(key);
    for (int i = 0; i < len; i++) {
        data[i] = data[i] ^ key[i % key_len];
    }
}
```

The modulo operator (%) allows "wrapping" around the key: when reaching the end of the key, we start over from the beginning.

Example with `"SECRET"` *(6 characters) on* `"FORMAT C: /Y"` *(12 characters):*

- *Position 0: 'F'* $\oplus$ *'S'*
- *Position 1: 'O'* $\oplus$ *'E'*
- ...
- *Position 6: ' '* $\oplus$ *'S' (back to key start)*

4. **Antivirus perspective:**

   (a) **Why signature detection is ineffective:**

   Each virus copy uses a different, randomly generated key. The encrypted code therefore produces a different byte sequence each time.

   Example with payload `"FORMAT C: /Y"`*:*

   - *With key 0x42:* `04 0D 10 0F 03 16 62 01 78 62 6D 1B`
   - *With key 0x5A:* `1C 15 08 17 1B 0E 7A 19 60 7A 75 03`

Impossible to create a unique signature to detect all variants.

(b) ***Alternative detection techniques:***

- ***Emulation / Sandboxing:*** *Execute the code in a controlled environment to observe its behavior after decryption*
- ***Decryptor detection:*** *The decryption routine (unencrypted part) has recognizable patterns (XOR loop)*
- ***Heuristic analysis:*** *Detect code sequences characteristic of XOR decryption*
- ***Entropy analysis:*** *Encrypted code has high entropy (quasi-random byte distribution)*

Key Takeaway

- *XOR is **self-inverse**: applying the same operation twice with the same key returns the original value ($A \oplus K \oplus K = A$).*
- ***Polymorphism** makes each virus copy unique in binary, preventing signature-based detection.*
- *With a single-byte key, there are only 255 possible variants (key 0x00 = no encryption). Brute-force is trivial.*
- *The **Vernam cipher** (One-Time Pad) is the only system proven to be unbreakable, but it requires a key as long as the message, random, and single-use.*

Exercise 3 (Companion Virus and PATH Hijacking)**.** A **companion virus** is a type of malware that does not modify existing files but places itself alongside them to be executed in their place. A common technique involves exploiting the `PATH` environment variable to intercept the execution of system commands.

Attack principle: When a user types a command (for example `ls`), the system searches for the executable in the directories listed in `PATH`, in order. If an attacker places a fake `ls` in a higher-priority directory, this malicious program will be executed instead.

Consider the following C program, designed to replace the `ls` command:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    // Phase 1: Malicious code (simulation)
    FILE *f = fopen("/tmp/log.txt", "a");
    if (f) {
        fprintf(f, "Intercepted command: ls\n");
        fprintf(f, "Directory: %s\n", getcwd(NULL, 0));
        fclose(f);
    }

    // Phase 2: Execute the real command
    execvp("/bin/ls", argv);

    return 0;
}
```

1. Explain how this program works. Why doesn't the user notice that their system is compromised?
2. How could an attacker install this companion virus on a Linux system? Propose two different methods.
3. What happens if the attacker forgets the line `execvp("/bin/ls", argv)`? What consequences would this have on the stealthiness of the attack?
4. Propose two countermeasures that the user or system administrator could implement to protect against this type of attack.
5. **Bonus:** Modify the program to also record the arguments passed to the `ls` command.

Course Reminder

Command resolution via PATH:
When a user types a command without an absolute path (e.g., `ls` instead of `/bin/ls`), the shell searches for the executable in the directories listed in the `PATH` environment variable, **in order**.

Typical PATH example on Linux:

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin
```

The shell searches in order: `/usr/local/bin`, then `/usr/bin`, then `/bin`, etc. The **first** executable found is the one that gets launched.

The `execvp()` function:

- **Replaces** the current process with a new program (the calling process ceases to exist)
- The `v` means arguments are passed as an array (`argv`)
- The `p` means the program is searched for in PATH (unless an absolute path is given)
- Unlike `system()`, `execvp()` does not create a subprocess (no implicit `fork`) and does not go through an intermediate shell

Difference between `execvp()` and `system()`:

	`execvp()`	`system()`
Process	Replaces the process	Creates a subprocess
Return	Does not return (if successful)	Returns the exit code
Shell	No intermediate shell	Goes through `/bin/sh`
Security	Safer (no shell injection)	Vulnerable to injection

Answer 3. 1. ***Program operation:***

The program executes in two phases:

Phase 1 – Malicious code:

- *Opens file `/tmp/log.txt` in append mode (`"a"`)*

- *Records that the `ls` command was intercepted*
- *Records the current directory (activity spying)*
- *Closes the file*

Phase 2 – Cover:

- *`execvp("/bin/ls", argv)` replaces the current process with the real `ls`*
- *Original arguments are fully transmitted*
- *The displayed result is identical to the real command*

Why it's stealthy: *The user sees the normal `ls` output. The malicious phase is invisible because it produces no screen output.*

2. ***Installation methods:***

Method 1: User PATH modification

```
# In ~/.bashrc or ~/.profile
export PATH="$HOME/.malware:$PATH"
# Then place fake ls in ~/.malware/ls
```

Method 2: Using current directory

```
# If PATH contains "." first
export PATH=".:$PATH"
# Place fake ls in victim's working directory
```

Other methods:

- *Exploit a vulnerability to write to `/usr/local/bin` (often before `/bin` in PATH)*
- *Create an alias in `.bashrc`: `alias ls='/path/malware'`*

Why you should never put "." in PATH

*If the current directory (`.`) is in PATH (especially in the first position), an attacker can place a malicious executable named `ls`, `cat`, or `sudo` in **any directory** visited by the victim (e.g., `/tmp`, a shared folder). The victim only needs to run a common command in that directory to launch the malware.*
*This is why modern Linux distributions **never** include `.` in PATH by default. On Windows, the current directory is searched first, which has historically caused numerous vulnerabilities (DLL hijacking, etc.).*

3. ***Without the execvp() line:***

- *The real `ls` command is never executed*
- *The user sees no output (or just return code 0)*
- *The user immediately notices something is wrong*
- *They will investigate and probably discover the malware*
- ***Stealthiness is completely compromised***

This is why the companion virus technique always requires executing the original program after the malicious code.

4. ***Countermeasures:***

   Countermeasure 1: Use absolute paths

```
alias ls='/bin/ls'
alias cat='/bin/cat'
# Or directly use /bin/ls instead of ls
```

   Countermeasure 2: Secure the PATH

   - *Never put "." in PATH*
   - *Regularly check PATH contents*
   - *Ensure priority directories are write-protected*

   Other countermeasures:

   - *Enable SELinux or AppArmor*
   - *Monitor modifications to* `.bashrc` *and* `.profile`
   - *Use* `type -a ls` *to see all available* `ls` *commands*

5. ***Bonus – Recording arguments:***

```
FILE *f = fopen("/tmp/log.txt", "a");
if (f) {
    fprintf(f, "Intercepted command: ls\n");
    fprintf(f, "Directory: %s\n", getcwd(NULL, 0));
    fprintf(f, "Arguments (%d):\n", argc);
    for (int i = 0; i < argc; i++) {
        fprintf(f, "  argv[%d]: %s\n", i, argv[i]);
    }
    fclose(f);
}
```

Key Takeaway

- *A* ***companion virus*** *does not modify any existing file: it places itself "alongside" to be executed instead of the legitimate program.*
- *The search order in* ***PATH*** *is critical: the first directory has priority.*
- *The* `execvp()` *function replaces the current process, ensuring transparency for the user.*
- ***Never*** *include* `.` *(current directory) in PATH.*
- *The* `type -a <cmd>` *command allows checking all locations of a command.*

Exercise 4 (Target File Search Routine)**.** Before being able to infect files, a virus must first locate them on the system. This **target search** phase is crucial: it must be efficient to maximize propagation while remaining stealthy to avoid detection.

Consider the following C program that recursively traverses a directory looking for files with a specific extension:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

void search_targets(const char *path, const char *extension) {
    DIR *dir = opendir(path);
    if (dir == NULL) return;

    struct dirent *entry;
    char fullpath[1024];

    while ((entry = readdir(dir)) != NULL) {
        // Ignore . and ..
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;

        snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry->d_name);

        struct stat st;
        if (stat(fullpath, &st) == -1) continue;

        if (S_ISDIR(st.st_mode)) {
            // Recurse into subdirectories
            search_targets(fullpath, extension);
        } else if (S_ISREG(st.st_mode)) {
            // Check the extension
            char *ext = strrchr(entry->d_name, '.');
            if (ext && strcmp(ext, extension) == 0) {
                printf("Target found: %s (%ld bytes)\n",
                       fullpath, st.st_size);
            }
        }
    }
    closedir(dir);
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <directory> <extension>\n", argv[0]);
        return 1;
    }

    printf("Searching for %s files in %s...\n", argv[2], argv[1]);
    search_targets(argv[1], argv[2]);

    return 0;
}
```

1. Analyze the code and explain the role of the functions `opendir()`, `readdir()`, `stat()`, and `closedir()`.

2. Why is it important to ignore the "." and ".." entries when traversing a directory? What would happen otherwise?

3. Modify the `search_targets()` function to search for executable files (having execution permission) instead of filtering by extension.

4. Recursive traversal can be problematic on large file systems. Propose a modification to limit the recursion depth to a maximum of N levels.

5. From an antivirus perspective, what suspicious behaviors could this type of program trigger? How could a behavioral antivirus detect this activity?

Course Reminder

File system traversal in C (POSIX):
Under Unix/Linux, a directory is a special file that contains a list of entries (name → inode). Traversal is done via the POSIX API: `opendir()`, `readdir()`, `closedir()`.

Special entries in every directory:

- `.` (dot): reference to the directory itself
- `..` (two dots): reference to the parent directory

File type test macros (defined in `<sys/stat.h>`):

Macro	Meaning
`S_ISREG(m)`	Regular file
`S_ISDIR(m)`	Directory
`S_ISLNK(m)`	Symbolic link
`S_ISCHR(m)`	Character device
`S_ISBLK(m)`	Block device
`S_ISFIFO(m)`	Named pipe (FIFO)
`S_ISSOCK(m)`	Socket

Execution permission bits:

Constant	Octal value	Meaning
`S_IXUSR`	0100	Execution by owner (`-x---`)
`S_IXGRP`	0010	Execution by group (`---x--`)
`S_IXOTH`	0001	Execution by others (`----x`)

Unix octal permissions reminder:

Octal	Binary	Permission
0	000	None
1	001	Execute (`x`)
2	010	Write (`w`)
3	011	Write + Execute (`wx`)
4	100	Read (`r`)
5	101	Read + Execute (`rx`)
6	110	Read + Write (`rw`)
7	111	Read + Write + Execute (`rwx`)

Example: `chmod 755 file` → `rwxr-xr-x` (owner: all, group and others: read + execute).

Answer 4. *1. **Role of functions:***

- *`opendir(path)`: Opens a directory and returns a `DIR*` pointer for traversal. Returns `NULL` on error (non-existent directory, insufficient permissions).*
- *`readdir(dir)`: Reads the next directory entry. Returns a pointer to a `struct dirent` containing the filename (`d_name`). Returns `NULL` when all entries have been read.*

- *`stat(path, &st)`: Gets file metadata (size, permissions, type, dates). Fills the `struct stat` passed as parameter. Returns -1 on error.*
- *`closedir(dir)`: Closes the directory and frees resources allocated by `opendir()`. Important to avoid file descriptor leaks.*

2. ***Importance of ignoring "." and "..":***

   - *`"."` represents the current directory*
   - *`".."` represents the parent directory*

   Without this filtering:

   - *With `"."`: The program would enter the same directory indefinitely → **infinite loop***
   - *With `".."`: The program would go up to root then back down → infinite traversal of the entire system*
   - *In both cases: **stack overflow** due to infinite recursion*

   > ***Warning***
   >
   > *Forgetting to filter "." and ".." is a classic programming error during recursive directory traversal. The program enters infinite recursion and ends with a segmentation fault (stack overflow). It is also a risk for a virus that could self-infect indefinitely or crash before being able to propagate.*

3. ***Search by execution permissions:***

```c
void search_targets(const char *path) {
    DIR *dir = opendir(path);
    if (dir == NULL) return;

    struct dirent *entry;
    char fullpath[1024];

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;

        snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry->d_name);

        struct stat st;
        if (stat(fullpath, &st) == -1) continue;

        if (S_ISDIR(st.st_mode)) {
            search_targets(fullpath);
        } else if (S_ISREG(st.st_mode)) {
            // Check execution permission (owner)
            if (st.st_mode & S_IXUSR) {
                printf("Executable found: %s (%ld bytes)\n",
                       fullpath, st.st_size);
            }
        }
```

```
    }
    closedir(dir);
}
```

Note: *You can also check* `S_IXGRP` *(group) and* `S_IXOTH` *(others) for a more complete search:*

```
if (st.st_mode & (S_IXUSR | S_IXGRP | S_IXOTH))
```

4. ***Limiting recursion depth:***

```
void search_targets(const char *path, const char *ext,
                    int depth, int max_depth) {
    if (depth > max_depth) return;  // Limit reached

    DIR *dir = opendir(path);
    if (dir == NULL) return;

    struct dirent *entry;
    char fullpath[1024];

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;

        snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry
    ->d_name);

        struct stat st;
        if (stat(fullpath, &st) == -1) continue;

        if (S_ISDIR(st.st_mode)) {
            // Increment depth for subdirectories
            search_targets(fullpath, ext, depth + 1, max_depth);
        } else if (S_ISREG(st.st_mode)) {
            char *e = strrchr(entry->d_name, '.');
            if (e && strcmp(e, ext) == 0) {
                printf("Target: %s\n", fullpath);
            }
        }
    }
    closedir(dir);
}

// Initial call with depth=0
search_targets("/home", ".c", 0, 3);  // Max 3 levels
```

5. ***Behavioral antivirus detection:***

   Detectable suspicious behaviors:

   - *Rapid, massive file tree traversal*
   - *Read access to numerous executable files*
   - *Characteristic pattern: many* `opendir/readdir/stat` *calls*

- *Access to sensitive directories (`/bin`, `/usr/bin`)*

Detection techniques:

- ***Rate limiting:*** *Alert if a process opens too many directories per second*
- ***Access pattern analysis:*** *Detect systematic traversal by extension*
- ***System call monitoring:*** *Count `stat()` calls on executables*
- ***Honeypots:*** *Place "bait" files and alert if accessed*

Key Takeaway

- *Recursive directory traversal uses the trio `opendir()` / `readdir()` / `closedir()`.*
- ***Always*** *filter "." and ".." to avoid infinite loops.*
- *The macros `S_ISDIR`, `S_ISREG` test the file type; the constants `S_IXUSR/S_IXGRP/S_IXOTH` test permissions.*
- *An efficient virus limits its search depth to remain stealthy and avoid crashes.*

Exercise 5 (Code Injection by Appending)**.** **Appending infection** is a classic technique where the virus adds its code to the end of a target file. To avoid infecting the same file multiple times (which could corrupt it or make it suspicious), the virus uses an **infection marker**.

Consider the following C program that simulates this mechanism on text files:

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MARKER "[[INFECTED]]"
#define PAYLOAD "\n--- MALICIOUS CODE SIMULATION ---\n"

int is_infected(const char *filename) {
    FILE *f = fopen(filename, "r");
    if (f == NULL) return -1;

    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), f) != NULL) {
        if (strstr(buffer, MARKER) != NULL) {
            fclose(f);
            return 1;  // Already infected
        }
    }
    fclose(f);
    return 0;  // Not infected
}

int infect_file(const char *filename) {
    if (is_infected(filename) == 1) {
        printf("[SKIP] %s is already infected\n", filename);
        return 0;
    }

```

```c
    FILE *f = fopen(filename, "a");
    if (f == NULL) {
        printf("[ERROR] Cannot open %s\n", filename);
        return -1;
    }

    fprintf(f, "%s", PAYLOAD);
    fprintf(f, "%s\n", MARKER);
    fclose(f);

    printf("[INFECTED] %s\n", filename);
    return 1;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <file1> [file2] ...\n", argv[0]);
        return 1;
    }

    for (int i = 1; i < argc; i++) {
        infect_file(argv[i]);
    }
    return 0;
}
```

1. Explain the role of the infection marker (`MARKER`). What would happen if the virus did not use a marker?
2. Why is the file opened in "a" (append) mode in the `infect_file()` function? What would be the effect of using "w" mode instead?
3. The `is_infected()` function scans the entire file line by line. Propose a more efficient method.
4. The marker `[[INFECTED]]` is easily detectable by an antivirus. Propose a modification to make the marker less visible.
5. From an antivirus perspective, what characteristics of this program could be used to detect it?

Course Reminder

File infection techniques:
A virus can infect an executable file in three main ways:

1. **Overwriting:** The virus replaces the beginning of the file with its own code. Simple but **destructive**: the original program is destroyed and no longer works. Immediate detection by the user.
2. **Appending:** The virus adds its code to the **end** of the file and modifies the entry point to execute itself first, then hands control back to the original program. The file remains functional.
3. **Prepending:** The virus inserts its code at the **beginning** of the file and shifts the original code. More complex but the virus executes first naturally.

Textual diagram – appending infection:

```
BEFORE infection:              AFTER infection:
+--------------------+         +--------------------+
| Original           |         | Original           |
| program            |         | program            |
|                    |         |                    |
+--------------------+         +--------------------+
                               | Virus code         |
                               | (payload)          |
                               +--------------------+
                               | Marker             |
                               | [[INFECTED]]       |
                               +--------------------+
```

The original program remains intact, the virus is appended after it. The marker at the end of the file prevents reinfection.

Answer 5. 1. ***Role of the infection marker:***

The marker identifies already infected files to ***avoid reinfection****.*

Without a marker, problems:

- *The file grows with each virus execution (payload added multiple times)*
- *File size becomes abnormally large → easy detection*
- *The file may become corrupted or unusable*
- *Degraded performance (increasingly long files)*
- *Risk of filling the hard drive*

The marker is placed at the end of the infected file for quick verification.

2. ***"a" mode vs "w" mode:***

Comparison table of file opening modes in C:

Mode	***Read***	***Write***	***Initial position***	***Existing file***
`"r"`	*Yes*	*No*	*Beginning*	*Must exist*
`"w"`	*No*	*Yes*	*Beginning*	***Truncated / Created***
`"a"`	*No*	*Yes*	*End*	*Preserved / Created*
`"r+"`	*Yes*	*Yes*	*Beginning*	*Must exist*
`"w+"`	*Yes*	*Yes*	*Beginning*	***Truncated / Created***
`"a+"`	*Yes*	*Yes*	*End*	*Preserved / Created*

"a" (append) mode:

- *Positions the cursor at the end of the file*
- *Writes are appended after the existing content*
- *The original content is* ***preserved***
- *The infected program/file remains functional*

"w" (write) mode:

- *Erases all existing content*
- *The file is* ***emptied*** *then rewritten from the beginning*
- *The original content is* ***destroyed***
- *The infected program no longer works at all*

A virus must preserve infected file functionality to:

- *Remain stealthy (the user notices nothing)*
- *Enable propagation (the program must execute)*

3. ***Optimizing is_infected():***

   Since the marker is always added at the end of the file, we can read directly from the end using `fseek()`*:*

```c
int is_infected(const char *filename) {
    FILE *f = fopen(filename, "r");
    if (f == NULL) return -1;

    // Position at end minus marker size
    int marker_len = strlen(MARKER) + 2;  // +2 for possible \n

    if (fseek(f, -marker_len, SEEK_END) == 0) {
        char buffer[64];
        if (fread(buffer, 1, marker_len, f) > 0) {
            buffer[marker_len] = '\0';
            if (strstr(buffer, MARKER) != NULL) {
                fclose(f);
                return 1;  // Infected
            }
        }
    }

```

```
    fclose(f);
    return 0;  // Not infected
}
```

Explanation of `fseek()`:

- *`fseek(f, offset, origin)` moves the read/write cursor in the file*
- *`SEEK_SET`: from the **beginning** of the file (offset ≥ 0)*
- *`SEEK_CUR`: from the **current position** (positive or negative offset)*
- *`SEEK_END`: from the **end** of the file (offset usually ≤ 0)*

Here, `fseek(f, -marker_len, SEEK_END)` positions the cursor `marker_len` bytes before the end of the file, allowing to read only the last bytes.

Advantages:

- *O(1) complexity instead of O(n)*
- *Very fast even for large files*
- *Fewer disk accesses*

4. ***Marker obfuscation:***

   Technique 1: XOR encryption

```
// Encrypted marker (key = 0x42)
#define MARKER_ENC "\x0e\x0e\x09\x0c\x04\x07\x25\x07\x04\x06\x0f\x0f"
// Decrypt before comparison
```

   Technique 2: File hash

   Instead of a textual marker, store a hash of the first bytes. If the hash matches, the file is infected.

   Technique 3: Marker in metadata

   Use file extended attributes (xattr) or modify the modification date in a characteristic way.

   Technique 4: Non-printable characters

```
#define MARKER "\x01\x02\x03\x04\x05\x06\x07\x08"
```

5. ***Antivirus detection techniques:***

   - ***Marker signature:*** *Search for the string `[[INFECTED]]` in files*
   - ***File access monitoring:*** *Detect append-mode opening patterns on many files*
   - ***Executable integrity:*** *Compare file hashes with a reference database (modification = alert)*
   - ***Growth analysis:*** *Alert if executable size increases suspiciously*
   - ***Behavioral analysis:*** *Detect a program that reads then writes to many executable files*
   - ***Static analysis:*** *Search for characteristic code patterns (fopen mode "a" + fprintf on variable files)*

Key Takeaway

- *The* ***infection marker*** *prevents reinfection and ensures that files remain functional.*
- *The* `"a"` *(append) mode is essential: it preserves the original file content, unlike* `"w"` *mode which destroys it.*
- `fseek(f, -n, SEEK_END)` *allows reading the last* n *bytes of a file in* $O(1)$*, a crucial optimization for infection checking.*
- *Marker obfuscation techniques (XOR, hash, metadata) complicate detection but do not prevent behavioral analysis.*